JU-6 Pi

6-VOICE POLYPHONIC SYNTHESIZER · RASPBERRY PI 5

OWNER'S MANUAL



Software version 1.1.0 · Played from an AKAI MPK mini Plus over USB · Audio via HDMI or Raspberry Pi DAC+

JU-6 Pi is an independent hobby project inspired by the Juno-106. It is not affiliated with or endorsed by Roland Corporation.

Contents

1 Introduction

2 Important Notes

3 Panel Descriptions

4 Connections

5 Power-up and Status LED

6 Playing

7 Selecting Patches

8 The Knob Pages

9 Creating Sounds

10 Writing (Saving) Patches

11 Audio Output and Latency

12 Settings File (ju6.conf)

13 MIDI Implementation

14 Parameter Chart

15 Patch List

16 Troubleshooting

17 Specifications

18 Sources and Credits

NOTE Everything in this manual about controls, MIDI numbers, patch memory, audio outputs and latency was taken from the JU-6 Pi v1.1.0 source code and README (repository `OleBee/juno106-pi`, commit `ffbce82`). Items that could not be confirmed are marked **NOT VERIFIED**.

1 Introduction

Thank you for building the **JU-6 Pi**. It turns a Raspberry Pi 5 (8 GB) into a stand-alone, six-voice polyphonic synthesizer in the spirit of the classic 1984 Juno-106. There is no screen, no network and no setup: write the SD card, plug in an AKAI MPK mini Plus and an audio output, switch on and play.

The sound is generated in real time by the **Ultramaster KR-106** DSP engine (GPL-3), a component-level model of the Juno-106 calibrated against measurements of real hardware. JU-6 Pi wraps that engine in its own headless real-time program (`ju6pi`, C++17, direct ALSA, no JACK or PipeWire). No ROM dumps or samples are used — every sound is computed.

Main features

- **6 voices** with Juno-106 style voice assignment (Poly I), plus Poly II and Unison.
- **DCO** with pulse (PWM from LFO, manual or ENV), sawtooth, sub-oscillator (square, one octave down), noise; RANGE 16' / 8' / 4'.
- **VCF** — 4-pole resonant low-pass that self-oscillates, with ENV (±polarity), LFO and keyboard follow; 2× or 4× oversampled.
- **HPF** with four positions; position 0 is the 106's bass boost.
- **VCA** switchable between ENV and GATE, with LEVEL. One **ADSR envelope** and a triangle **LFO** with RATE and DELAY.
- **Chorus** I, II and I+II, modelled as a bucket-brigade (BBD) circuit with companding and noise.
- **128 patch memories** (A11–B88) pre-loaded with the factory sounds, stored in genuine Juno-106 SysEx format.
- Four **knob pages** give the MPK's eight knobs access to every sound parameter.
- Read-only system on the SD card — you may pull the power at any time.

To get the most out of the instrument, please read this manual from beginning to end. Keep it handy for later reference.

The DSP engine follows the Juno-106 closely, but JU-6 Pi is *not* a Roland product. "Roland" and "Juno" are trademarks of their respective owners and are used here only to describe what the synthesizer emulates.

2 Important Notes

Power supply

- Use the official Raspberry Pi **27 W USB-C** power supply (recommended in the project README). The Pi also powers the MPK mini Plus through its USB port.
- The root file system is read-only (overlayroot). The only thing the synthesizer writes to is the **JU6DATA** partition, and it writes atomically (new file, fsync, rename). It is therefore safe to switch off simply by pulling the power — there is no shut-down procedure.

Placement and handling

- Use the Pi with adequate cooling (the project is built around a Pi 5 with the **Active Cooler**; the audio thread keeps one CPU core busy and the CPU is held at full clock).
- When fitting a DAC+ board, check that no GPIO pin is shifted by one row **before** applying power.
- Do not connect or disconnect the DAC+ board while the Pi is powered.

Listening level

- Turn down the amplifier, TV or monitors before switching on. Two short **ready tones** are played about 10–15 seconds after power-up.
- The VCF can self-oscillate at high RES settings and the HPF bass boost (position 0) adds low end. Keep the volume moderate while experimenting.

Memory backup

- Your patches are stored in `ju6/patches.syx` and `ju6/names.txt` on the JU6DATA partition. Put the SD card in a computer from time to time and copy these files to a safe place.
- Deleting `patches.syx` and `names.txt` restores the factory sounds.

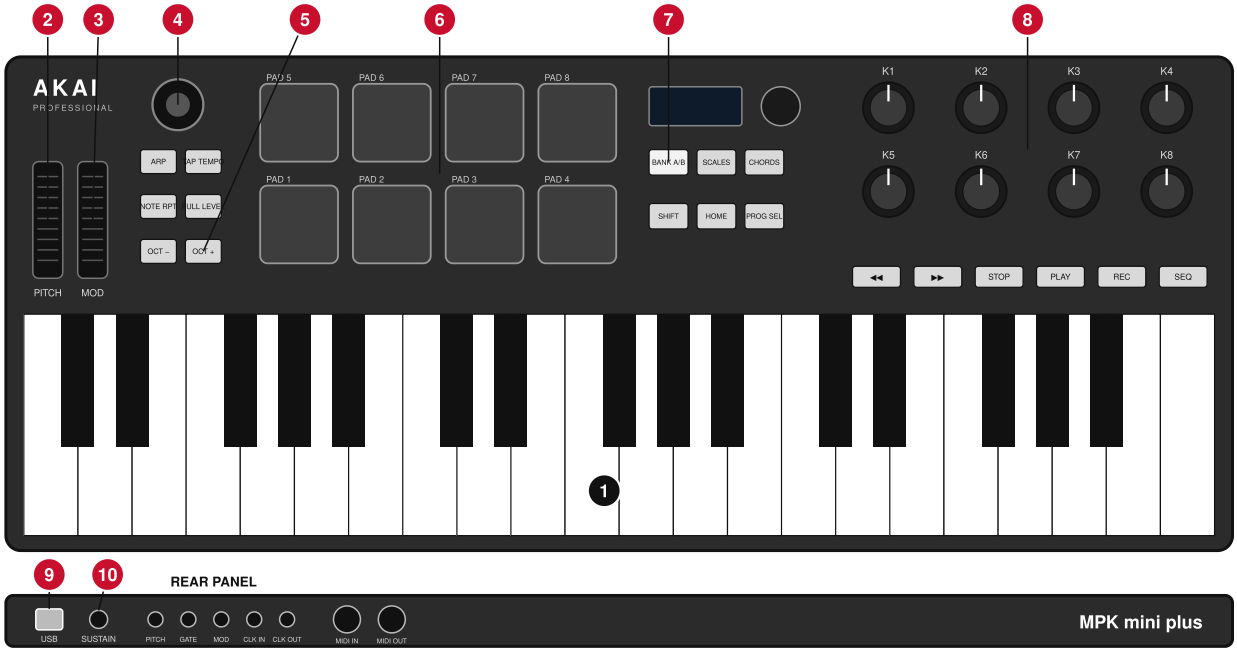
What is not on board

For a clean, fast boot the image has Wi-Fi, Bluetooth, SSH, NetworkManager, cloud-init, avahi, cron, apt timers and swap switched off. The system log is kept in RAM only. All configuration is done by editing a text file on the SD card (see section 12).

> **CAUTION** This manual describes software version **1.1.0**. The audio-output selection (`audio_output`) and DAC+ support do not exist in 1.0.0.

3 Panel Descriptions

JU-6 Pi has no panel of its own — the **AKAI MPK mini Plus** is its front panel. The numbers below refer to the drawing; only controls that JU-6 Pi uses are numbered.

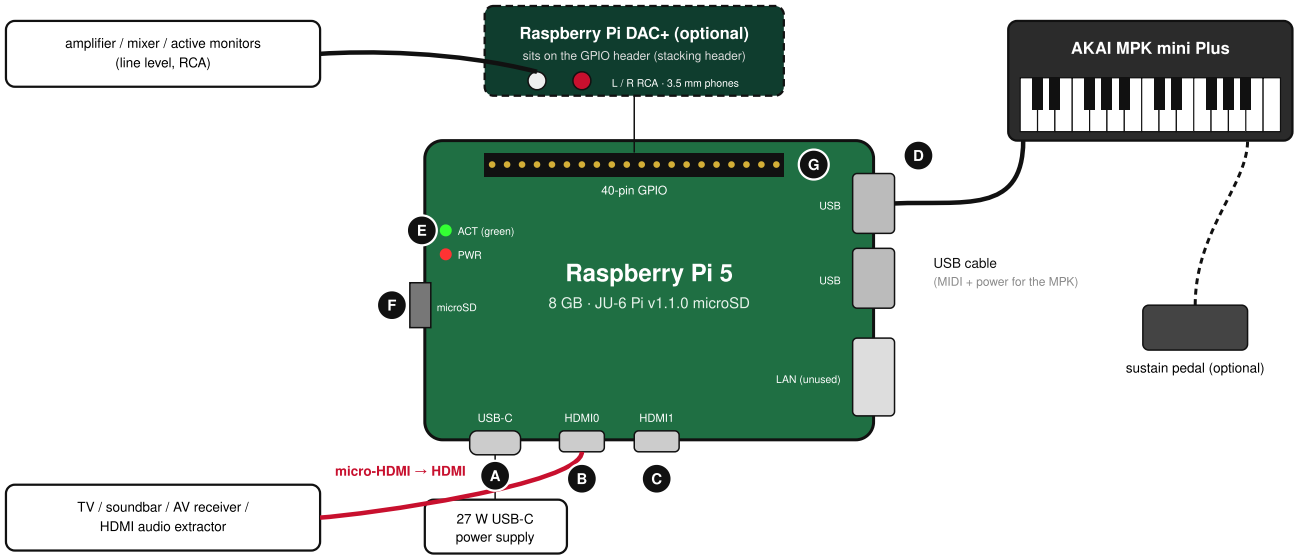


AKAI MPK mini Plus, top and rear panel (drawing based on the Top/Rear Panel figures in AKAI's MPK mini Plus User Guide v1.2; simplified, not to scale). Note that pads 1–4 are the **lower** row and pads 5–8 the **upper** row.

1. **Keyboard** — 37 keys. Plays JU-6 Pi on every MIDI channel (omni). Velocity is ignored by default, like the original. Also used to set the key with TRANSPOSE (pad bank B, pad 8).
2. **Pitch wheel** — BENDER. Bends the DCO (default depth 0.25 ≈ ±2 semitones) and optionally the VCF. Depths: knob page 4, K1/K2.
3. **Modulation wheel** (CC 1) — MOD: LFO vibrato on the DCO. Depth follows how far the wheel is moved (scaled by page 4, K3). In LFO MANUAL mode it also starts the LFO.
4. **X-Y joystick** — JU-6 Pi reacts to whatever the joystick sends: **Pitch Bend** acts as BENDER, **CC 1** acts as MOD. Other CC numbers are ignored unless listed in `mod_cc`. *NOT VERIFIED: which messages the joystick sends in the MPK's default program.*
5. **OCT –** / **OCT +** — handled inside the MPK; shifts the keyboard range. Press both to return to the centre octave.
6. **Pads 1–8** — bank A: patch number 1–8 (hold 1.5 s to write). Bank B: page select, group –/+, HOLD, TRANSPOSE. See section 7.
7. **BANK A/B** — switches the pads between bank A (patches) and bank B (functions).
8. **Knobs K1–K8** (CC 70–77) — sound parameters. What each knob does depends on the selected *page* (section 8).
9. **USB port** — connect to any USB port on the Pi. Carries MIDI and powers the MPK.
10. **Sustain input** — a sustain pedal (CC 64) works as HOLD.

NOTE The other MPK functions (arpeggiator, scales, chords, note repeat, sequencer, transport buttons, CV/Gate, 5-pin MIDI) are run by the MPK itself. They are not used or tested by JU-6 Pi; any notes they produce over USB are simply played.

Raspberry Pi 5 (the sound module)



Simplified connection drawing (not to scale). The DAC+ board is optional.

A	**USB-C power** — 27 W supply recommended.
B	**HDMI0** (micro-HDMI, the port **nearest the USB-C power connector**) — primary HDMI audio output.
C	**HDMI1** — used automatically as a reserve if HDMI0 is not available.
D	**USB ports** — MPK mini Plus (any port). Other class-compliant USB-MIDI devices are connected automatically and can be hot-plugged. A USB audio interface can be used as an output.
E	**ACT LED (green)** — the synthesizer's status display (section 5).
F	**microSD slot** — the JU-6 Pi card (system + JU6DATA partition).
G	**40-pin GPIO header** — mount the optional Raspberry Pi DAC+ here (on a stacking header if you use the Active Cooler).

Raspberry Pi DAC+ (optional)

The **Raspberry Pi DAC+** (formerly IQaudIO DAC+) is a sound card that sits on the GPIO pins. It has a TI PCM5122 converter, two RCA line outputs (0–2 V RMS) and a 3.5 mm headphone output carrying the same stereo signal. It has no input. Older black IQaudIO DAC+ boards work with the same overlay.

> **NOT VERIFIED** ***HiFiBerry DAC+ is not supported in v1.1.0.*** *JU-6 Pi 1.1.0 loads only the `rpi-dacplus` overlay and recognises only the ALSA cards `RPiDAC` / `IQaudIODAC` (`src/audio_out.h`, `image/boot/dacplus.blokk`). A HiFiBerry DAC+ uses a different overlay and card name and is **not** detected by this version; JU-6 Pi would then play on HDMI instead. This manual therefore describes the Raspberry Pi DAC+.*

4 Connections

CAUTION To prevent damage, turn the volume of your amplifier or TV all the way down and leave the Pi unpowered while making connections.

Preparing the SD card

1. Download `ju6pi-1.1.0.img.xz` from the project's Releases page. If it is split into `.part-aa`, `.part-ab` … join the parts first (`cat ju6pi-1.1.0.img.xz.part-* > ju6pi-1.1.0.img.xz`) and check it with `sha256sum -c ju6pi-1.1.0.img.xz.sha256`.
2. Write it to a microSD card (8 GB or more). **Raspberry Pi Imager:** Choose Device → Raspberry Pi 5; Choose OS → Use Custom → the .img.xz file; Choose Storage → the card. When asked about OS customisation, answer **No**. **balenaEtcher:** Flash from file → Select target → Flash!
3. Insert the card into the Pi.

Connecting the equipment

1. **Audio, with DAC+:** connect the RCA outputs to an amplifier, mixer or active monitors. The synthesizer uses the board automatically when it is fitted.
2. **Audio, without DAC+:** connect **HDMI0** — the micro-HDMI port nearest the USB-C power connector — to a TV, soundbar, AV receiver or HDMI audio extractor. No screen is needed; the image supplies its own EDID and forces stereo PCM at 48 kHz on both ports.
3. **Controller:** connect the AKAI MPK mini Plus to any USB port on the Pi.
4. Optionally connect a sustain pedal to the MPK's SUSTAIN input.
5. Connect the USB-C power supply last.

Mounting the DAC+ in a KKSB case with Active Cooler

1. Fit the Active Cooler and connect its cable to the FAN connector.
2. Press the 40-pin stacking header supplied with the case onto the GPIO pins; it lifts the DAC above the cooler.
3. Fit the DAC+ on top with its RCA sockets towards the case openings, and secure it with the supplied standoffs and screws.
4. Check that no pin is offset by a row before switching on.

NOTE An HDMI audio extractor typically adds 0–2 ms. TVs and AV receivers can add 20–100 ms of their own processing — choose a **Game**, **Direct** or **Pure** mode on such devices.

5 Power-up and Status LED

1. Turn the amplifier or TV volume down.
2. Connect the power supply. The green ACT LED blinks slowly while the system starts.
3. After about **10–15 seconds** two short **rising** ready tones (A–E) are heard and the LED lights steadily. The synthesizer is now ready.
4. Raise the volume gradually while playing.

The last selected patch is recalled at power-up (it is remembered in `ju6/state.txt`; on the very first start, `start_patch` decides). The program is a systemd service that restarts automatically within one second if it ever stops.

Status LED (green ACT LED on the Pi)

	**Slow blinking (1 Hz)**	Starting up, or waiting for an audio output.
	**Steady on**	Ready to play.
	**On, two short gaps every 3 s**	Ready, but on a **reserve output**: `audio_output = dacplus` is set and the DAC+ was not found.
	**On, one short gap every 2 s**	Ready, but **no MIDI device** found — check the MPK's USB cable.
	**Rapid blinks**	Page or group changed (number of blinks = page or group number), HOLD switched, or a patch written (5 blinks).

If the ready tones are **falling** (E–A) instead of rising, the synthesizer is playing on a reserve output because the DAC+ you asked for was not found (see section 11).

Checking without a screen

Every few seconds the synthesizer writes `ju6/status.txt` on the JU6DATA partition: software version, chosen audio output, sound cards found, buffer size and latency, VCF oversampling, number of MIDI devices and underrun count. Put the SD card in a computer to read it. With DAC+ it should contain lines like:

```
lyd: Raspberry Pi DAC+ (RCA/hodetelefon) (plughw:CARD=RPiDAC,DEV=0)
lydkort funnet: vc4hdmi0, vc4hdmi1, RPiDAC
```

The status file is written in Norwegian: *lyd* = audio, *lydkort funnet* = sound cards found, *ADVARSEL* = warning.

6 Playing

Keyboard

The keyboard plays on all MIDI channels (`key_channel = omni`); only the pad notes on the pad channel are reserved. **Velocity is off** by default, as on the original; switch it on with page 4, K7 or `velocity = on`. Use the MPK's OCT –/+ buttons to reach higher and lower octaves.

BENDER (pitch wheel / joystick X)

Pitch Bend bends the DCO pitch by up to ±2 semitones (default `bender_dco = 0.25`) and, if set, the VCF cutoff (default `bender_vcf = 0`). Change the depths on page 4 with K1 (→ DCO) and K2 (→ VCF).

MOD (mod wheel / CC 1)

MOD applies LFO vibrato to the DCO. Unlike the 106's spring-loaded lever with fixed depth, MOD here is **continuous**: the depth follows how far the wheel or joystick is pushed, scaled by page 4, K3 (default `mod_depth = 0.5`). When the LFO is in MANUAL mode (page 3, K8), moving MOD away from zero also triggers the LFO.

HOLD

A sustain pedal (CC 64) holds the notes while pressed. Pad 7 in bank B switches HOLD on and off as a latch (LED: 2 blinks = on, 1 blink = off).

TRANSPOSE

1. Press and **hold pad 8** (bank B).
2. Press a key. That key becomes the new tonic: **C4 (middle C, note 60) = no transposition**; F4 gives +5 semitones, for example. The key itself does not sound.
3. Release the pad.

A **short press** (under 0.6 s, no key) on pad 8 switches the transposition off — and the next short press brings back the last value. Page 4, K4 sets the transposition directly from –12 to +12 semitones (centre = 0).

Voice modes

Mode	Page 4, K6 zone	Description
Poly I (default)	0–42	Juno-106 voice assignment, as on the hardware.
Poly II	43–85	Alternative polyphonic assignment (an addition — not on the 106).
Unison	86–127	All voices on one note (an addition — not on the 106).

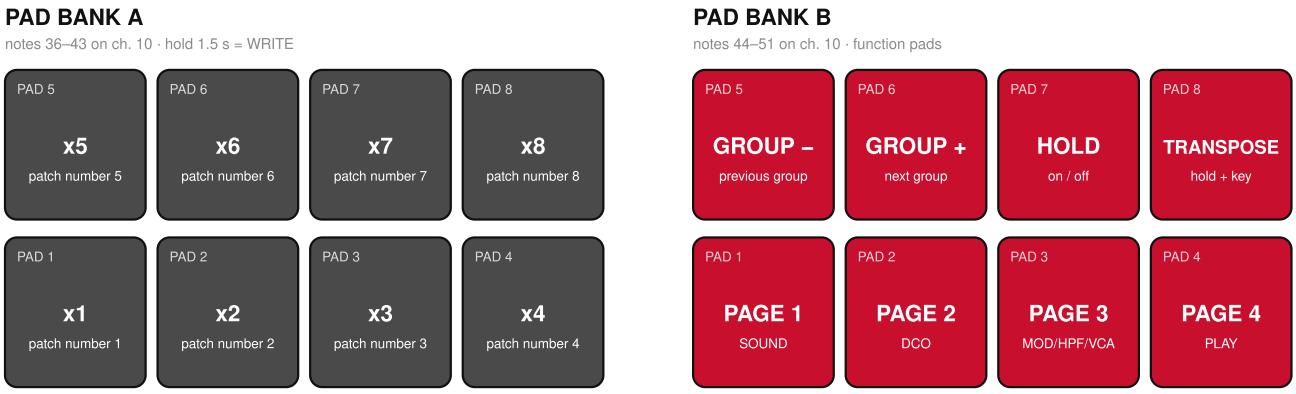
PORTAMENTO (page 4, K8) adds glide; it is 0 (off) by default. Neither portamento, Unison nor velocity exist on the original instrument.

Other controls

MIDI CC 7 sets the master volume (as does page 4, K5). CC 120 and 123 silence all notes; CC 121 resets bender and MOD.

7 Selecting Patches

As on the 106, the 128 memories are organised as two **banks** (A and B), each with eight **groups** of eight **patches**: A11 … A88 and B11 … B88. The first digit is the group, the second the patch number.



Pad functions. Press the MPK's **BANK A/B** button to switch the pad row. Pads 1–4 are the lower row.

Selecting a patch

1. Press BANK A/B so the pads are in **bank B**.
2. Press pad 5 (**GROUP –**) or pad 6 (**GROUP +**) to step through the 16 groups A1 → A8 → B1 → B8 (it wraps round). The LED blinks the group number (1–8) and the patch with the same number in the new group is selected.
3. Press BANK A/B to return to **bank A**.
4. Press pad 1–8 to select patch 1–8 in that group. The sound changes immediately.

Example: group A4, pad 8 → **A48 Synth Bass I**.

> **NOTE** The LED shows only the group number, not whether you are in bank A or B. Starting from group A1, GROUP + seven times reaches A8; once more reaches B1.

Program Change

MIDI Program Change 0–127 selects A11 … B88 directly (0 = A11, 63 = A88, 64 = B11, 127 = B88). The MPK can send program changes from its pads (pad Play Mode "ProgCh" in the MPK's program editor, per AKAI's guide).

SysEx

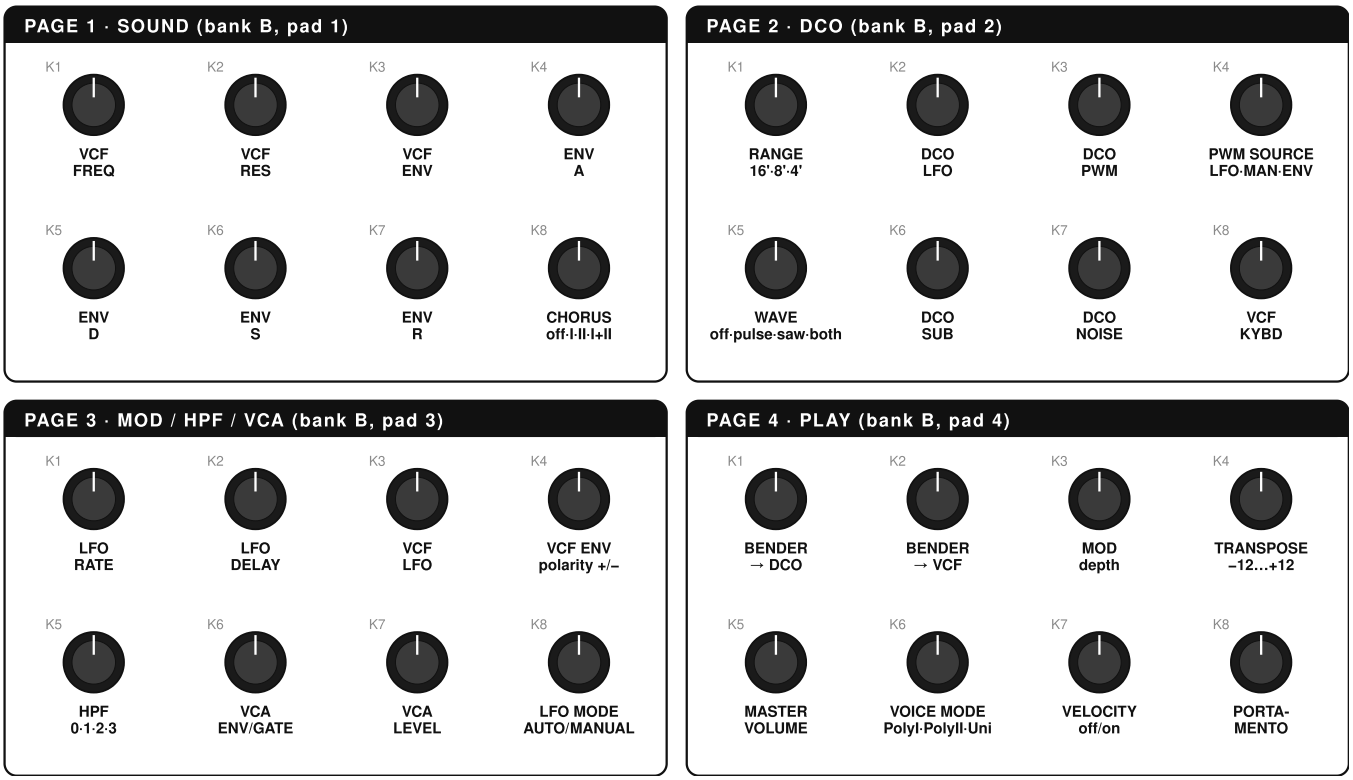
A Juno-106 patch dump (APR, `F0 41 30 0n …`) or single-parameter message (IPR, `F0 41 32 0n …`) received over MIDI is loaded into the current sound, just as on a 106. This lets you audition sounds from Juno-106 editors and librarians.

Master volume across patches

Selecting a patch changes every sound parameter but keeps the master volume (page 4, K5) where it was.

8 The Knob Pages

The eight knobs send **CC 70–77**. Their function depends on the selected **page**. Page 1 is active after power-up; select a page with pads 1–4 in **bank B** (the LED blinks the page number).



Switch-type knobs

Knobs that replace a switch divide the knob's 0–127 travel into equal zones:

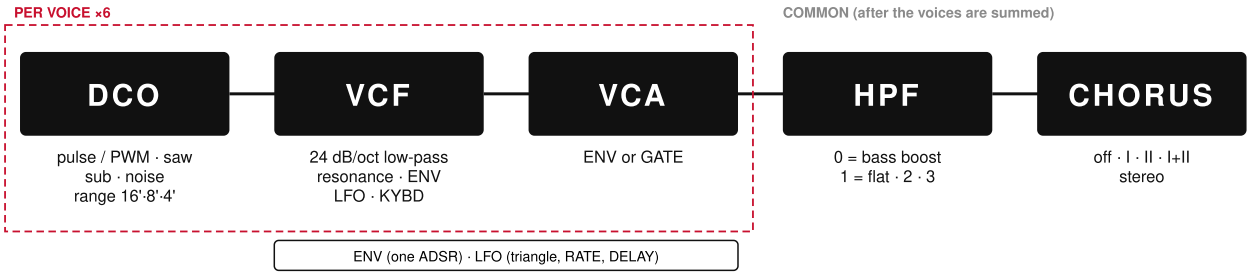
Knob	Zones (value sent by the knob)
P1 K8 CHORUS	0–31 off · 32–63 I · 64–95 II · 96–127 I+II
P2 K1 RANGE	0–42 16' · 43–85 8' · 86–127 4'
P2 K4 PWM SOURCE	0–42 LFO · 43–85 MAN · 86–127 ENV
P2 K5 WAVE	0–31 off · 32–63 pulse · 64–95 saw · 96–127 pulse + saw
P2 K6 SUB	0 = sub off; 1–127 = sub level
P3 K4 VCF ENV polarity	0–63 + (normal) · 64–127 – (inverted)
P3 K5 HPF	0–31 pos. 0 (bass boost) · 32–63 pos. 1 (flat) · 64–95 pos. 2 · 96–127 pos. 3
P3 K6 VCA	0–63 ENV · 64–127 GATE
P3 K8 LFO MODE	0–63 AUTO · 64–127 MANUAL (triggered by MOD)
P4 K4 TRANSPOSE	25 zones, –12 … +12 semitones (centre ≈ 64 = 0)
P4 K6 VOICE MODE	0–42 Poly I · 43–85 Poly II · 86–127 Unison
P4 K7 VELOCITY	0–63 off · 64–127 on

Knob behaviour

By default the knobs **jump**: the parameter takes the knob's value as soon as the knob is moved (`knob_takeover = jump`). With `knob_takeover = pickup` a knob does nothing until it passes the current value (soft takeover; this resets on every page or patch change). With `knob_mode = relative` JU-6 Pi reads the knobs as relative (two's complement: 1–63 = up, 65–127 = down). *NOT VERIFIED: that this matches the MPK mini Plus's own "Relative" knob mode.*

9 Creating Sounds

Each of the six voices has its own DCO, VCF and VCA. The voices are summed and pass through the common HPF and chorus. A single envelope (ENV) and a single LFO control all voices.



Section	What it does	Where
DCO	The sound source. Pulse (width set by PWM, modulated by LFO, set manually or by ENV), sawtooth, sub-oscillator one octave down, and noise are mixed. RANGE sets the octave; DCO LFO adds vibrato.	Page 2
VCF	Low-pass filter, 24 dB/octave. FREQ = cutoff, RES = resonance (self-oscillates near maximum), ENV = how much the envelope opens the filter (polarity on P3 K4), LFO = filter wobble, KYBD = cutoff follows the keyboard.	Pages 1–3
VCA	Loudness. ENV: follows the ADSR. GATE: full level while a key is held. LEVEL sets the patch volume (about –10 … +10 dB).	Page 3
ENV	One ADSR envelope with Juno-106 curves: A = attack, D = decay, S = sustain level, R = release.	Page 1
LFO	Triangle wave. RATE = speed, DELAY = fade-in time after a key is pressed (AUTO), or triggered by MOD (MANUAL).	Page 3
HPF	Position 0 = bass boost (about +9.4 dB low shelf), 1 = flat, 2 ≈ 236 Hz, 3 ≈ 754 Hz high-pass.	P3 K5
Chorus	I = slow, wide; II = faster, deeper; I+II = a third, distinct mode (fast, shallow, vibrato-like).	P1 K8

How to use the tutorials

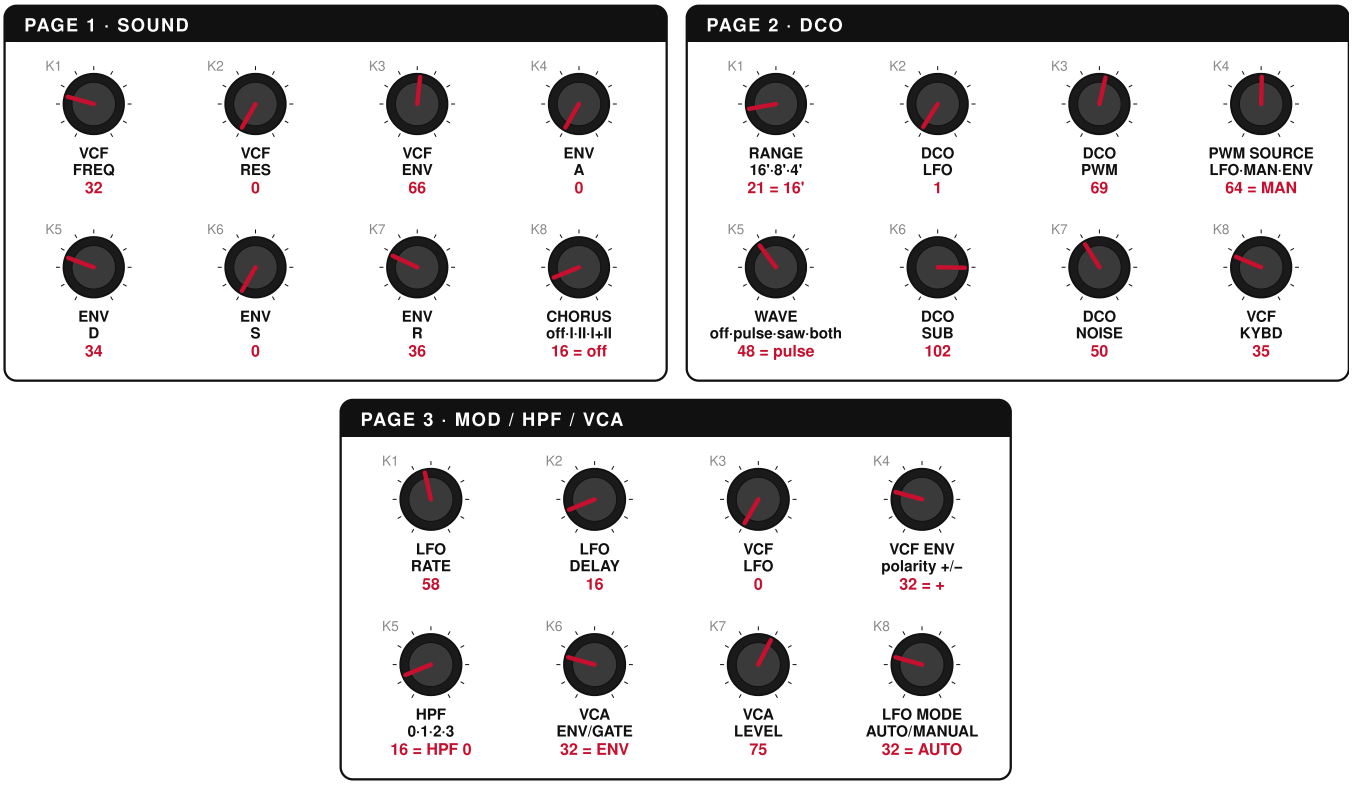
There is no "initialise patch" function, so every tutorial lists the value for **all 24 knobs** on pages 1–3. Work page by page: select the page with its pad in bank B, then turn each knob until it sends the target value (0–127). The patches are re-creations of factory sounds, so you can press the original's pad at any time to compare — but note that this replaces what you have dialled in.

> **NOTE** Start from a patch in a group you don't mind experimenting in. Because the knobs jump by default, each knob must actually be turned at least a little so it sends its value.

Tutorial 1 — Synth bass (A48 Synth Bass I)

A punchy, mono-style bass made from a narrow pulse, a strong sub-oscillator and a short filter envelope.

1. Page 2: set RANGE to 16' (K1 = 21) and WAVE to pulse only (K5 = 48). Set PWM SOURCE to MAN (K4 = 64) and PWM to 69 for a fairly narrow pulse.
2. Page 2: add the sub-oscillator (K6 = 102) for weight and a touch of NOISE (K7 = 50) for attack. VCF KYBD 35.
3. Page 1: close the filter (FREQ 32, RES 0) and let the envelope open it (VCF ENV 66).
4. Page 1: envelope A = 0, D = 34, S = 0, R = 36 — a plucked shape with no sustain. CHORUS off (K8 = 16).
5. Page 3: HPF position 0 (K5 = 16) for the bass boost, VCA = ENV (K6 = 32), LEVEL 75. LFO RATE 58 / DELAY 16 / VCF LFO 0 (barely used).
6. Play low on the keyboard. Try more RES or VCF ENV for a squelchier bass.



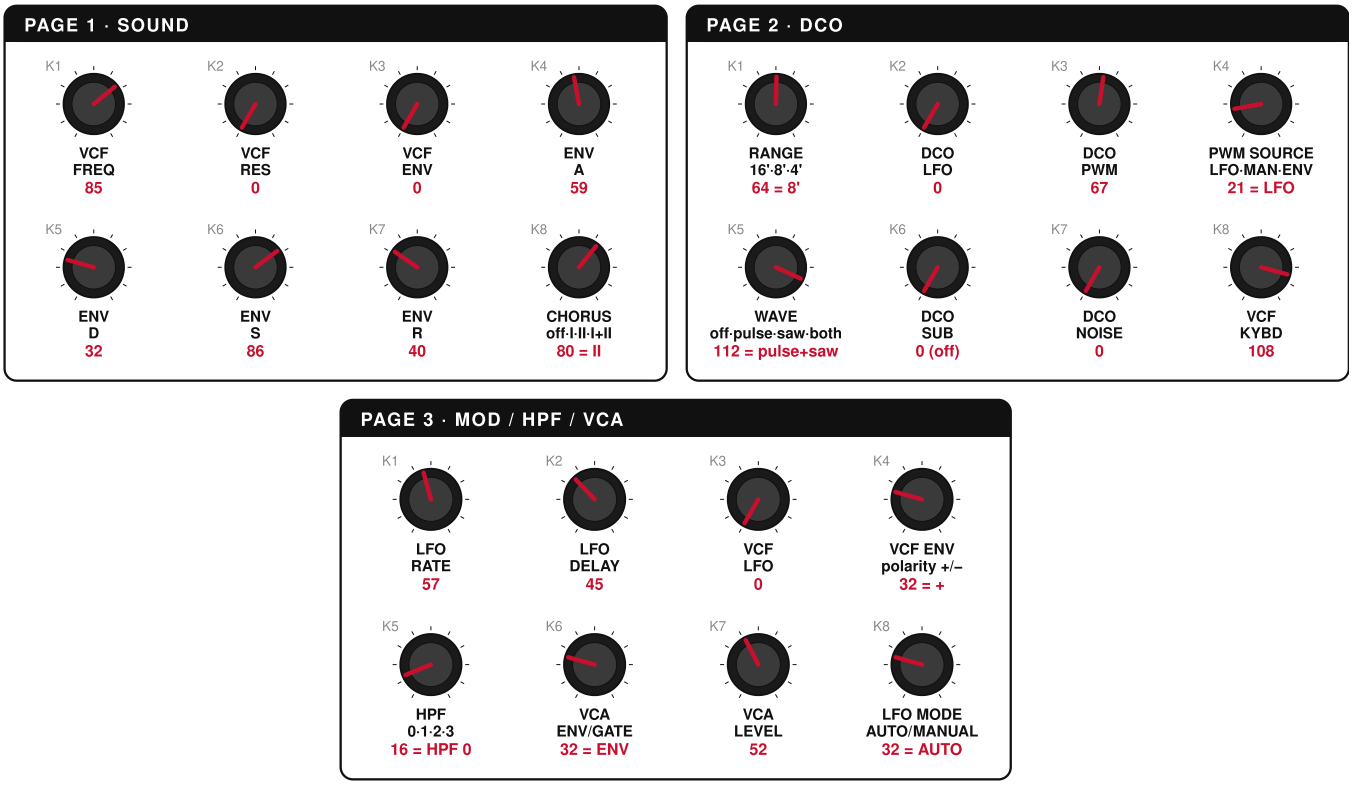
	K1	K2	K3	K4	K5	K6	K7	K8
Page 1	VCF FREQ **32**	VCF RES **0**	VCF ENV **66**	ENV A **0**	ENV D **34**	ENV S **0**	ENV R **36**	CHORUS off·I·II·I+II **16 = off**
Page 2	RANGE 16'·8'·4' **21 = 16'**	DCO LFO **1**	DCO PWM **69**	PWM SOURCE LFO·MAN·ENV **64 = MAN**	WAVE off·pulse·saw·both **48 = pulse**	DCO SUB **102**	DCO NOISE **50**	VCF KYBD **35**
Page 3	LFO RATE **58**	LFO DELAY **16**	VCF LFO **0**	VCF ENV polarity +/– **32 = +**	HPF 0·1·2·3 **16 = HPF 0**	VCA ENV/GATE **32 = ENV**	VCA LEVEL **75**	LFO MODE AUTO/MANUAL **32 = AUTO**

Target knob values (0–127) for **A48 Synth Bass I**, computed from the factory patch data in `src/factory_bank.h` with the same scaling the synth uses when it loads a patch. Red pointer = approximate knob position on a 0–127 scale.

Tutorial 2 — Strings (B11 Strings)

A classic ensemble: pulse and saw together, slowly swept by the LFO, softened by the envelope and widened by chorus II.

1. Page 2: RANGE 8' (K1 = 64), WAVE pulse + saw (K5 = 112). PWM SOURCE LFO (K4 = 21) with PWM 67 — the LFO sweeps the pulse width for movement. SUB 0 (off), NOISE 0, VCF KYBD 108.
2. Page 3: LFO RATE 57, DELAY 45, VCF LFO 0. HPF position 0 (K5 = 16). VCA ENV (K6 = 32), LEVEL 52.
3. Page 1: open the filter fairly far (FREQ 85), RES 0, VCF ENV 0 — the tone stays constant.
4. Page 1: slow envelope: A = 59, D = 32, S = 86, R = 40.
5. Page 1: CHORUS II (K8 = 80). Compare with I (48) and I+II (112).
6. Play chords. Raise ENV A for slower bowing, or ENV R for longer tails.



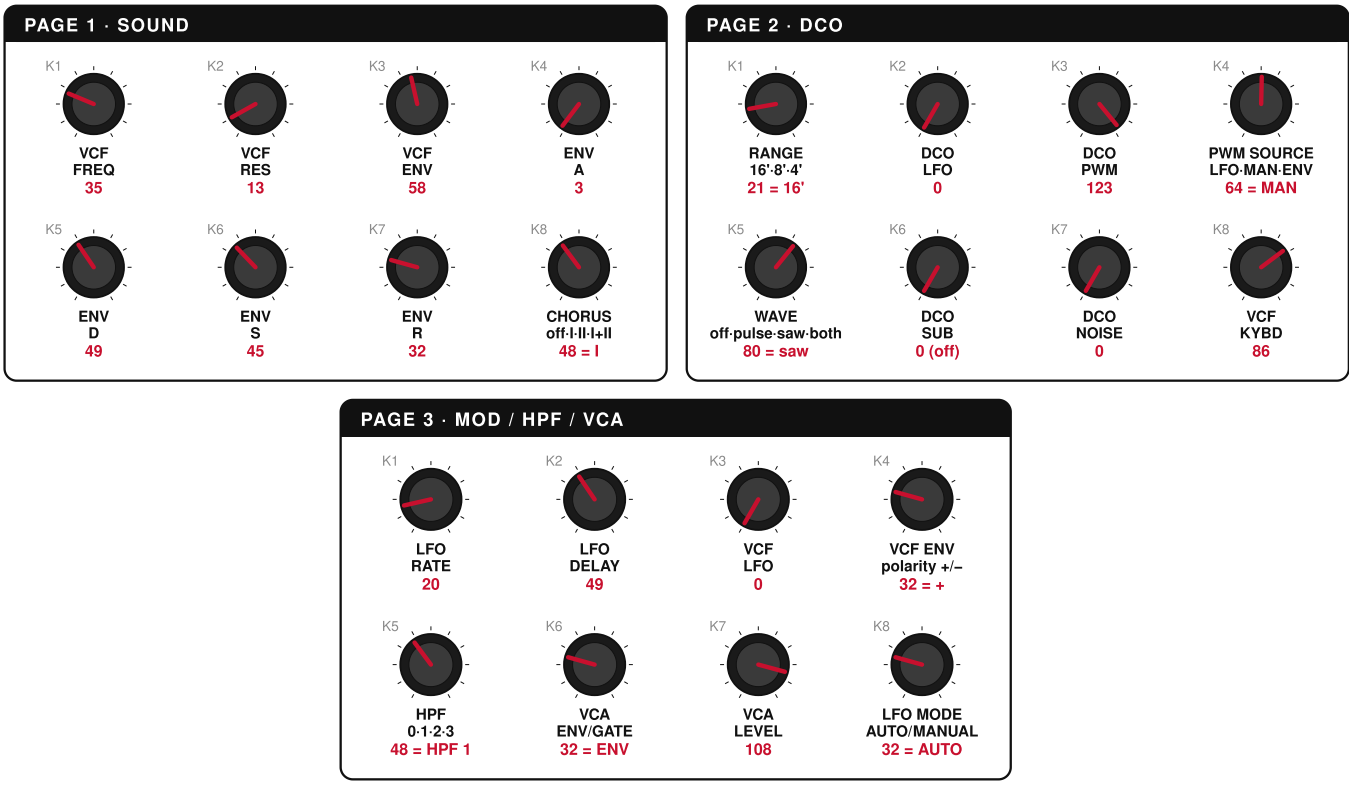
	K1	K2	K3	K4	K5	K6	K7	K8
Page 1	VCF FREQ **85**	VCF RES **0**	VCF ENV **0**	ENV A **59**	ENV D **32**	ENV S **86**	ENV R **40**	CHORUS off·I·II·I+II **80 = II**
Page 2	RANGE 16'·8'·4' **64 = 8'**	DCO LFO **0**	DCO PWM **67**	PWM SOURCE LFO·MAN·ENV **21 = LFO**	WAVE off·pulse·saw·both **112 = pulse+saw**	DCO SUB **0 (off)**	DCO NOISE **0**	VCF KYBD **108**
Page 3	LFO RATE **57**	LFO DELAY **45**	VCF LFO **0**	VCF ENV polarity +/– **32 = +**	HPF 0·1·2·3 **16 = HPF 0**	VCA ENV/GATE **32 = ENV**	VCA LEVEL **52**	LFO MODE AUTO/MANUAL **32 = AUTO**

Target knob values (0–127) for **B11 Strings**, computed from the factory patch data in `src/factory_bank.h` with the same scaling the synth uses when it loads a patch. Red pointer = approximate knob position on a 0–127 scale.

Tutorial 3 — Brass (A11 Brass)

Synth brass relies on a sawtooth and a filter that opens with the envelope and settles back, giving the 'blat' of a horn attack.

1. Page 2: RANGE 16' (K1 = 21), WAVE saw only (K5 = 80). PWM SOURCE MAN (K4 = 64); PWM 123 has no audible effect while the pulse is off. SUB 0, NOISE 0, VCF KYBD 86.
2. Page 1: FREQ 35, RES 13 and a generous VCF ENV 58 — the envelope does the work.
3. Page 1: envelope A = 3, D = 49, S = 45, R = 32: fast attack, the filter falls back to a sustained level.
4. Page 1: CHORUS I (K8 = 48).
5. Page 3: LFO RATE 20, DELAY 49, VCF LFO 0. HPF position 1 = flat (K5 = 48). VCA ENV (K6 = 32), LEVEL 108.
6. Use the mod wheel for delayed vibrato on held notes. For a slower swell, raise ENV A to about 60.



	K1	K2	K3	K4	K5	K6	K7	K8
Page 1	VCF FREQ **35**	VCF RES **13**	VCF ENV **58**	ENV A **3**	ENV D **49**	ENV S **45**	ENV R **32**	CHORUS off·I·II·I+II **48 = I**
Page 2	RANGE 16'·8'·4' **21 = 16'**	DCO LFO **0**	DCO PWM **123**	PWM SOURCE LFO·MAN·ENV **64 = MAN**	WAVE off·pulse·saw·both **80 = saw**	DCO SUB **0 (off)**	DCO NOISE **0**	VCF KYBD **86**
Page 3	LFO RATE **20**	LFO DELAY **49**	VCF LFO **0**	VCF ENV polarity +/– **32 = +**	HPF 0·1·2·3 **48 = HPF 1**	VCA ENV/GATE **32 = ENV**	VCA LEVEL **108**	LFO MODE AUTO/MANUAL **32 = AUTO**

Target knob values (0–127) for **A11 Brass**, computed from the factory patch data in `src/factory_bank.h` with the same scaling the synth uses when it loads a patch. Red pointer = approximate knob position on a 0–127 scale.

10 Writing (Saving) Patches

As implemented in v1.1.0, the write procedure is:

1. Select the target group with GROUP –/+ (bank B, pads 5/6).
2. Switch the pads to bank A and **hold** the pad of the target patch number for **1.5 seconds** (`save_hold_seconds`).
3. The LED blinks rapidly to confirm. The memory is written to `ju6/patches.syx` and named "*A15 Egen*" ("Egen" = user) in `ju6/names.txt`.

KNOWN ISSUE (v1.1.0) Reading `src/controller.h` shows that pressing a bank-A pad **first recalls the patch stored on that pad** (`selectSlot()` on note-on), and the save 1.5 s later captures the sound *as it is then*. Edits made before pressing the pad are therefore replaced by the stored patch and are **not** written; only knob changes made while the pad is held (within the 1.5 s) end up in memory. The built-in self-test checks only that the save happens to the right slot, not that edits are kept. Until this is fixed in the software, do not rely on the write function to keep a sound you have edited — write the knob values down (the tutorial pages can be used as patch sheets).

Patch files on the JU6DATA partition

File	Contents
`ju6/patches.syx`	All 128 memories as standard Juno-106 SysEx (APR) messages; created the first time you write. Can be opened in Juno-106 editors.
`ju6/names.txt`	One line per user patch: `A11<TAB>name[<TAB>ext]`. *ext* bit 0 = chorus I+II, bit 1 = PWM from ENV.
`ju6/fabrikk-A-B.syx`	Copy of factory banks A and B for reference.
`ju6/state.txt`	Last selected patch and buffer setting (automatic).
`ju6/status.txt`	What the synth found at the last start-up.
`ju6/ju6.conf`	Your settings (section 12).

Juno-106 format compatibility

The Juno-106 patch format has no room for chorus **I+II** or **PWM from ENV**. JU-6 Pi stores these in the extra column of `names.txt`, so they come back on JU-6 Pi; a real 106 or an editor sees the nearest valid setting (chorus I, PWM MAN).

Restoring factory sounds

Delete `patches.syx` and `names.txt` from the JU6DATA partition. Deleting `state.txt` resets the last-patch and buffer memory.

11 Audio Output and Latency

The output is chosen with `audio_output` in `ju6/ju6.conf`. Cards are found by name, not number, so ALSA's card order doesn't matter.

`audio_output`	Behaviour
`auto` (default)	Raspberry Pi DAC+ if fitted, otherwise HDMI0, then HDMI1, then a USB audio interface.
`dacplus`	Raspberry Pi DAC+. If it is missing, plays on HDMI0/HDMI1/USB but warns: falling ready tones, LED with two short gaps every 3 s, and an `ADVARSEL` (warning) line in `status.txt`.
`hdmi`	Only HDMI0, then HDMI1 — exactly as in 1.0.0. The DAC+ is ignored even if fitted.
`usb`	Only a USB audio interface.
`null`	No audio (testing).

At start-up the synth waits up to `dacplus_wait` = 8 s for the DAC driver when the board's HAT EEPROM reports a DAC or `audio_output = dacplus`. If it started on HDMI and the DAC+ appears within the first minute, it switches over and plays the ready tones again there. DAC+ output is 48 kHz, 32-bit; the mixer is set to 0 dB at start-up (2 V RMS). Lower it with `dacplus_volume_db` (0 or negative). HDMI output is 2-channel PCM at 48 kHz.

Buffer sizes (48 kHz)

The synth starts with the smallest buffer and moves up one step automatically after repeated underruns (3 or more within 30 s). The step is remembered for the next start.

Step	HDMI	HDMI buffer	DAC+	DAC+ buffer	USB	USB buffer
1 (start)	256 × 2	**10.7 ms**	128 × 2	**5.3 ms**	64 × 2	**2.7 ms**
2	256 × 3	16.0 ms	128 × 3	8.0 ms	128 × 2	5.3 ms
3	512 × 2	21.3 ms	256 × 2	10.7 ms	128 × 3	8.0 ms
4	512 × 3	32.0 ms	256 × 3	16.0 ms	256 × 2	10.7 ms
5	1024 × 2	42.7 ms	512 × 2	21.3 ms	256 × 3	16.0 ms
6	1024 × 3	64.0 ms	512 × 3	32.0 ms	512 × 2	21.3 ms

Period × number of periods, from `src/audio_out.h`. A fixed size can be forced with e.g. `buffer = 512x2`.

Key-to-sound latency

Estimated as USB-MIDI (≈1 ms) + one period of processing + the buffer + the receiving device:

Output (step 1)	Estimated latency out of the Pi
Raspberry Pi DAC+ (RCA)	≈ 8–10 ms
HDMI	≈ 12–17 ms, plus the receiver (extractor 0–2 ms; TV/AVR 20–100 ms)
USB audio interface (step 2)	≈ 7–10 ms total

NOT VERIFIED *These latency figures are **calculated, not measured** on hardware — the project README states that none of them has been measured yet. The actual output and buffer chosen are reported in `ju6/status.txt`.*

Real-time setup

The audio thread runs with SCHED_FIFO priority 80, locked memory, pinned to CPU 3, which is isolated from other processes (`isolcpus=3`). The CPU governor is fixed at *performance* and USB autosuspend is off so the controller never sleeps. VCF oversampling is chosen at start-up: 4× if the CPU manages it with a good margin (≥ 2.5× real time), otherwise 2×.

12 Settings File (ju6.conf)

All settings live in `ju6/ju6.conf` on the **JU6DATA** partition (FAT, 256 MB), which any computer can read. Every line is commented out; remove the `#` in front of a line and change the value. No recompiling is needed — the new settings apply at the next power-up.

Key	Default	Meaning
`audio_output`	auto	auto, dacplus, hdmi, usb or null (section 11)
`dacplus_volume_db`	0	DAC+ “Digital” mixer level in dB (0 or negative)
`dacplus_wait`	8	seconds to wait for the DAC+ at start-up
`audio_devices`	(not set)	legacy priority list (dacplus hdmi0 hdmi1 usb null); used only without audio_output
`buffer`	auto	auto, or fixed PERIODxCOUNT, e.g. 512x2
`oversample`	auto	VCF oversampling 2 or 4
`chime`	on	ready tones
`output_gain_db`	0	extra output level
`voices`	6	number of voices (1–10)
`voice_mode`	1	1 = Poly I, 2 = Poly II, 0 = Unison
`velocity`	off	velocity sensitivity
`bender_dco / bender_vcf`	0.25 / 0.0	bender depths (0–1)
`mod_depth`	0.5	MOD depth
`master_volume`	0.5	volume
`start_patch`	0	patch at the very first start (afterwards the last one is remembered)
`key_channel`	omni	keyboard channel (omni or 1–16)
`pad_channel`	10	pad channel
`knob_cc`	70 … 77	CC numbers for K1–K8
`pad_notes_a / pad_notes_b`	36 … 43 / 44 … 51	pad notes for bank A / bank B
`mod_cc`	1	CCs that drive MOD (several may be listed)
`knob_mode`	absolute	or relative
`knob_takeover`	jump	or pickup
`save_hold_seconds`	1.5	how long a pad is held to write

If your MPK sends different numbers

AKAI does not publish a factory-default table for the MPK mini Plus. JU-6 Pi assumes the MPK family's usual defaults: knobs CC 70–77, pads on MIDI channel 10 with notes 36–43 (bank A) and 44–51 (bank B), mod wheel CC 1, sustain CC 64. *NOT VERIFIED on the hardware for this manual.* If your unit differs, either

- change `knob_cc`, `pad_notes_a`, `pad_notes_b` and `pad_channel` in `ju6.conf`, or
- edit the MPK program (Shift + Prog Select, or AKAI's Program Editor) so the pads send those notes and the knobs those CCs in **Absolute** mode.

The three partitions

Partition	Contents	Mounted
bootfs (FAT)	firmware, kernel, config.txt, cmdline.txt	read-only
rootfs (ext4)	Raspberry Pi OS Lite (arm64, Debian 13 "trixie", 2026-10-06) + JU-6 Pi	read-only (overlayroot, changes in RAM)
JU6DATA (FAT, 256 MB)	ju6/ju6.conf, patches.syx, names.txt, state.txt, status.txt	read/write

13 MIDI Implementation

Message	Channel	Function
Note On / Off	omni (or `key_channel`)	play; velocity used only when VELOCITY is on
Note 36–43	10	pad bank A: patch 1–8 (hold 1.5 s = write)
Note 44–47	10	pad bank B 1–4: knob page 1–4
Note 48 / 49	10	pad bank B 5/6: group – / +
Note 50	10	pad bank B 7: HOLD on/off
Note 51	10	pad bank B 8: TRANSPOSE (hold + key; short press = on/off)
Pitch Bend	any	BENDER (DCO and VCF depth)
CC 1 (and other `mod_cc`)	any	MOD (LFO depth; LFO trigger in MANUAL)
CC 7	any	master volume
CC 64	any	HOLD (≥ 64 = on)
CC 70–77	any	knobs K1–K8, current page
CC 120, 123	any	all notes off
CC 121	any	reset bender and MOD
Program Change 0–127	any	select A11 … B88
SysEx `F0 41 30 0n pp …F7`	—	Juno-106 patch (APR) into the current sound
SysEx `F0 41 32 0n cc vv F7`	—	Juno-106 single parameter (IPR)

Pad notes are recognised only on the pad channel; notes on other channels (and pad-channel notes that are not pad notes) play the synth. JU-6 Pi transmits no MIDI.

Patch data format

Each patch is 18 bytes, as in the Juno-106 APR message: 16 slider values (0–127) in the order LFO RATE, LFO DELAY, DCO LFO, DCO PWM, DCO NOISE, VCF FREQ, VCF RES, VCF ENV, VCF LFO, VCF KYBD, VCA LEVEL, ENV A, ENV D, ENV S, ENV R, DCO SUB, followed by two switch bytes:

- **SW1:** bits 0–2 = range 16'/8'/4'; bit 3 = pulse; bit 4 = saw; bit 5 = chorus off (0 = on); bit 6 = chorus I (0 = II).
- **SW2:** bit 0 = PWM MAN (0 = LFO); bit 1 = VCF ENV inverted; bit 2 = VCA GATE (0 = ENV); bits 3–4 = HPF (3 – position).

14 Parameter Chart

Parameter	Page	Knob	MIDI	Range / positions	Stored in patch
VCF FREQ	1	K1	CC 70	0–127	slider 6
VCF RES	1	K2	CC 71	0–127	slider 7
VCF ENV	1	K3	CC 72	0–127	slider 8
ENV A / D / S / R	1	K4–K7	CC 73–76	0–127	sliders 12–15
CHORUS	1	K8	CC 77	off · I · II · I+II	SW1 b5, b6 (+ ext b0)
RANGE	2	K1	CC 70	16' · 8' · 4'	SW1 b0–2
DCO LFO	2	K2	CC 71	0–127	slider 3
DCO PWM	2	K3	CC 72	0–127	slider 4 (0–105 in 106 format)
PWM SOURCE	2	K4	CC 73	LFO · MAN · ENV	SW2 b0 (+ ext b1)
WAVE (pulse/saw)	2	K5	CC 74	off · pulse · saw · both	SW1 b3, b4
DCO SUB	2	K6	CC 75	0 = off, 1–127	slider 16
DCO NOISE	2	K7	CC 76	0–127	slider 5
VCF KYBD	2	K8	CC 77	0–127	slider 10
LFO RATE	3	K1	CC 70	0–127	slider 1
LFO DELAY	3	K2	CC 71	0–127	slider 2
VCF LFO	3	K3	CC 72	0–127	slider 9
VCF ENV polarity	3	K4	CC 73	+ · –	SW2 b1
HPF	3	K5	CC 74	0 · 1 · 2 · 3	SW2 b3–4
VCA mode	3	K6	CC 75	ENV · GATE	SW2 b2
VCA LEVEL	3	K7	CC 76	0–127 (≈ –10 … +10 dB)	slider 11
LFO MODE	3	K8	CC 77	AUTO · MANUAL	not stored
BENDER → DCO	4	K1	CC 70	0–127	not stored
BENDER → VCF	4	K2	CC 71	0–127	not stored
MOD depth	4	K3	CC 72	0–127	not stored
TRANSPOSE	4	K4	CC 73	–12 … +12	not stored
MASTER VOLUME	4	K5	CC 74	0–127	not stored
VOICE MODE	4	K6	CC 75	Poly I · Poly II · Unison	not stored
VELOCITY	4	K7	CC 76	off · on	not stored
PORTAMENTO	4	K8	CC 77	0–127	not stored

Page = knob page selected with bank B pads 1–4. Slider numbers refer to the 16 slider bytes of the patch data (section 13). "ext" = JU-6 Pi extension column in `names.txt`. Page-4 settings are performance settings and are not written with a patch; their start values come from `ju6.conf`.

Patch sheet

Patch	Page 1: FREQ · RES · ENV · A · D · S · R · CHORUS							
	Page 2: RANGE · DCO LFO · PWM · PWM SRC · WAVE · SUB · NOISE · KYBD							
	Page 3: LFO RATE · DELAY · VCF LFO · ENV POL · HPF · VCA · LEVEL · LFO MODE							

Copy this sheet to note down your own sounds.

15 Patch List

Banks A and B contain the Juno-106 factory patches as provided by the KR-106 project (parameter values, not samples), in exact 106 format (`patches/factory_106.syx`). Columns = patch number (pad 1–8), rows = group.

Bank A	1	2	3	4	5	6	7	8
A1x	Brass	Brass Swell	Trumpet	Flutes	Moving Strings	Brass & Strings	Choir	Piano I
A2x	Organ I	Organ II	Combo Organ	Calliope	Donald Pluck	Celeste	E Piano I	E Piano II
A3x	Clock Chimes	Steel Drums	Xylophone	Brass III	Fanfare	Strings III	Pizzicato	High Strings
A4x	Bass Clarinet	English Horn	Brass Ensemble	Guitar	Koto	Dark Pluck	Funky I	Synth Bass I
A5x	Lead I	Lead II	Lead III	Funky II	Synth Bass II	Funky III	Thud Wah	Going Up
A6x	Piano II	Clav.	Frontier Organ	Snare Drum	Tom Toms	Timpani	Shaker	Synth Pad
A7x	Sweep I	Pluck Sweep	Repeater	Sweep II	Pluck Bell	Dark Synth Piano	Sustainer	Wah Release
A8x	Gong	Resonance Funk	Drum Booms	Dust Storm	Rocket Men	Hand Claps	FX Sweep	Caverns

Bank B	1	2	3	4	5	6	7	8
B1x	Strings	Violin	Chorus Vibes	Organ I	Harpsichord I	Recorder	Perc Pluck	Noise Sweep
B2x	Space Chimes	Nylon Guitar	Orchestral Pad	Bright Pluck	Organ Bell	Accordion	FX Rise 1	FX Rise 2
B3x	Brass	Helicopter	Lute	Chorus Funk	Tomita	FX Sweep 1	Sharp Reed	Bass Pluck
B4x	Resonant Rise	Harpsichord 2	Dark Ensemble	Contact Wah	Noise Sweep 2	Glassy Wah	Phase Ensemble	Chorused Bell
B5x	Clav	Organ 2	Bassoon	Auto Noise Sweep	Brass Ensemble	Ethereal	Chorus Bell 2	Blizzard
B6x	E Piano Trem	Clarinet	Thunder	Reedy Organ	Flute / Horn	Toy Rhodes	Surf's Up	OW Bass
B7x	Piccolo	Melodic Taps	Meow Bass	Violin (high)	High Bells	Rolling Wah	Ping Bell	Brassy Organ
B8x	Low Dark Strings	Piccolo Trumpet	Cello	High Strings	Rocket Men	Forbidden Planet	Froggy	Owgan

Names from `src/factory_bank.h`. Memories you write are renamed "*xNN Egen*".

16 Troubleshooting

Symptom	Check
No ready tones, LED keeps blinking slowly	No audio output found. With HDMI, use **HDMI0** (nearest USB-C). Check the receiver's input. Read `ju6/status.txt`.
Ready tones fall (E–A) instead of rising	`audio_output = dacplus` but the DAC+ was not found — check that it is seated correctly. Output is on HDMI/USB meanwhile.
LED on with a short gap every 2 s	No MIDI device: check the MPK's USB cable. Devices can be plugged in while running.
Keys play but knobs/pads do nothing or do the wrong thing	The MPK program sends other CC/note numbers or another pad channel. Adjust `ju6.conf` or the MPK program (section 12). Make sure the pads are in the bank you expect (BANK A/B).
Pad presses play notes	Pads not on the pad channel (default 10) — set `pad_channel` to match.
Knobs have no effect until turned past a point	`knob_takeover = pickup` is set — this is soft takeover.
Noticeable delay with a TV or AV receiver	Select Game/Direct/Pure mode, or use a DAC+ or HDMI audio extractor.
Clicks or drop-outs	The buffer grows automatically after repeated underruns; you can also set `buffer = 512x2`.
Everything sounds transposed	TRANSPOSE active: short-press pad 8 (bank B), or set page 4, K4 to the centre.
Notes hang	Press HOLD (bank B, pad 7) to release, or check the sustain pedal.
An edited sound was not kept after writing	Known issue in v1.1.0 (section 10).
Want factory sounds back	Delete `patches.syx` and `names.txt` on JU6DATA.

With a screen and keyboard on the Pi

`ju6pi list` (patch names), `ju6pi bench` (CPU margin), `ju6pi demo DIR` (render WAV examples), `ju6pi utganger` (sound cards and chosen output), `ju6pi selftest` (MPK mapping, writing, transpose, SysEx, output selection), `journalctl -u ju6pi` (log). `aplay -l` should list `RPiDAC [RPi DAC+]` when the DAC+ is fitted.

17 Specifications

Sound engine	Ultramaster KR-106 DSP (component-level Juno-106 model), GPL-3, unmodified in third_party/kr106
Polyphony	6 voices (configurable 1–10); Poly I, Poly II, Unison
DCO	Pulse with PWM (LFO / MAN / ENV), sawtooth, sub-oscillator (square, –1 oct), noise; range 16' / 8' / 4'
VCF	4-pole resonant low-pass (IR3109 character), self-oscillating; ENV ±, LFO, KYBD; 2× or 4× oversampling
HPF	4 positions: 0 bass boost (≈ +9.4 dB shelf), 1 flat, 2 ≈ 236 Hz, 3 ≈ 754 Hz
VCA	ENV or GATE; LEVEL ≈ –10 … +10 dB
ENV / LFO	One ADSR (Juno-106 curves); triangle LFO with RATE and DELAY, AUTO or MANUAL trigger
Chorus	BBD model (MN3009 type), modes off / I / II / I+II, stereo
Memory	128 patches (A11–B88), Juno-106 SysEx (APR) format; factory banks A and B
Controller	AKAI MPK mini Plus over USB (any class-compliant USB-MIDI device, hot-plug)
Audio outputs	Raspberry Pi DAC+ (RCA + 3.5 mm, 48 kHz/32-bit) · HDMI0 / HDMI1 (2-ch PCM, 48 kHz) · USB audio
Sample rate	48 kHz
Buffer at start	DAC+ 5.3 ms · HDMI 10.7 ms · USB 2.7 ms (automatically increased on underruns)
Latency	≈ 8–10 ms (DAC+), ≈ 12–17 ms (HDMI) — calculated, not measured
Start-up time	≈ 10–15 s to ready tones
Hardware	Raspberry Pi 5, 8 GB; microSD ≥ 8 GB; 27 W USB-C supply recommended
Operating system	Raspberry Pi OS Lite arm64 (Debian 13 “trixie”), read-only root (overlayroot)
Software	ju6pi 1.1.0 (9 October 2026), C++17, ALSA; SCHED_FIFO 80 on isolated CPU 3
Licence	GPL-3.0

Specifications are taken from the v1.1.0 source and README and are subject to change.

NOTE Chorus II and I+II speeds have not yet been verified against hardware by the KR-106 project; the chorus circuit is modelled on the Juno-6/60 family, which shares its topology with the 106. Analogue voice-to-voice variation is only approximated.

18 Sources and Credits

What this manual is based on

Topic	Source (repository OleBee/juno106-pi, v1.1.0, commit ffbce82)
Knob pages, pad functions, CC/notes, write, transpose, Program Change	`src/controller.h`
Parameter scaling, patch format, MOD/bender, transpose key	`src/synth.h`, `third_party/kr106/DSP/KR106SysEx.h`, `KR106_DSP_SetParam.h`, `KR106_HPF.h`, `KR106LFO.h`
Defaults, LED, ready tones, status file, oversampling, buffer step-up	`src/main.cpp`, `image/data/ju6.conf`
Audio outputs, DAC+ detection, buffer tables	`src/audio_out.h`, `image/boot/dacplus.blokk`
Patch memory and files	`src/bank.h`, `src/factory_bank.h`, `image/data/LES-MEG.txt`
Installation, latency estimates, MPK defaults, changelog	`README.md`
MPK mini Plus layout and controls	AKAI Professional, *MPK mini Plus User Guide v1.2* (diagram redrawn)

Credits

- Sound engine: **Ultramaster KR-106** by Karl LaRocca and the original Ultramaster group (GPL-3).
- Raspberry Pi OS © Raspberry Pi Ltd. and the Debian project.
- AKAI Professional and MPK are trademarks of inMusic Brands. Diagrams in this manual are original drawings.
- The layout of this manual is inspired by 1980s synthesizer owner's manuals; no text has been copied.
- JU-6 Pi is not affiliated with Roland Corporation. "Roland" and "Juno" are trademarks of their owners.



JU-6 Pi · Owner's Manual · v1.1.0 · October 2026